



Implementation of a Data-Driven State-Feedback Controller for a Self-Stabilising Bicycle

Führungswerk

Vorgelegt an der Universität Stuttgart von
Hadi Elnemr

Master of Science Informationstechnik

Prüfern: Prof. Dr. -Ing. Andrea Iannelli,
Jun.-Prof. Dr.-Ing. Florian Pfaff
Betreuer: M.Sc. Yifan Xie

20. May 2025

Erklärung

Ich erkläre, die Arbeit selbständig verfasst und bei der Erstellung dieser Arbeit die einschlägigen Bestimmungen, insbesondere zum Urheberrechtsschutz fremder Beiträge, eingehalten zu haben. Soweit meine Arbeit fremde Beiträge (z. B. Bilder, Zeichnungen, Textpassagen) enthält, erkläre ich, dass diese Beiträge als solche gekennzeichnet sind (z.B. Zitat, Quellenangabe) und ich eventuell erforderlich gewordene Zustimmung der Urheber zur Nutzung dieser Beiträge in meiner Arbeit eingeholt habe.

Stuttgart, den 20.05.2025

(...)

Contents

List of Figures	VII
List of Tables	IX
1 Introduction	1
1.1 Bicycle Modelling	1
1.2 Bicycle Control	2
1.3 Data-Driven Control Methods	3
1.4 WHEELTEC MiniBike	3
1.5 Structure of thesis	4
2 Theory	5
2.1 Bicycle Modelling	5
2.2 Data-Driven Min-Max MPC	7
3 Simulations	11
3.1 Linear Quadratic Regulator (LQR)	13
3.2 Data-Driven State-Feedback Controller	14
3.3 Data-Driven Min-Max MPC Controller	15
3.4 Quantitative comparison of different controllers	16
4 Experimental Work	17
4.1 Mini-Bike	17
4.2 Experiments	18
5 Conclusion and Future work	22
5.1 Conclusion	22
5.2 Future Work	22
Bibliography	23

List of Figures

1.1	A picture for the MiniBike used in the experiments.	4
2.1	A Kinetic no-fork model of the bicycle	6
2.2	A fork model showing an inclination with angle λ of the steering axis, introducing a trail distance r_{tau}	7
3.1	Lean and steering angles over time under LQR controller with different Q and R settings and different system models, for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, and process noise bounded by $\epsilon = 10^{-6}$, and sampling period $T_s = 0.02s$	13
3.2	Lean and steering angles over time under different data-driven state-feedback settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, process noise bound $\epsilon = 10^{-6}$ and sampling period $T_s = 0.02s$ for different fork angle models.	15
3.3	Lean and steering angles over time under different data-driven MPC settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, noise bound $\epsilon = 10^{-6}$, sampling period $T_s = 0.02s$, and 150 MPC iterations for different fork angle models. . .	16
4.1	A picture for the MiniBike used in the experiments.	17
4.2	Lean angle over time under different data-driven state-feedback settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, and noise bound $\epsilon = 0.001$ for data synthesis of length $T = 40$. The data is smoothed using a rolling window which takes the mean of each 20 data points. The average lean angle achieved is also shown for each experiment.	21

List of Tables

3.1	Simulation Parameters	12
3.2	Comparison of LQR controller performance across different Q , R settings and fork models	14
3.3	Comparison of controllers across different fork models under $Q = \text{diag}(300, 0.1, 300)$, $R = 1$ and data length of $T = 120$	16
4.1	Overview of controllers applied on the mini-bike	19

Abstract

Riding a bicycle is a common activity for humans, but designing a controller to autonomously stabilise a bicycle presents significant engineering challenges. Traditional model-based controllers, such as linear quadratic regulators (LQR) and model predictive control (MPC), require an accurate system model. However, some parameters of a bicycle, i.e., the centre of gravity, can be difficult to determine and may fluctuate due to varying loads or additional passengers. In this research project, a data-driven state-feedback controller is implemented to stabilise an autonomous mini-bike by solving a semidefinite program (SDP). The method does not identify the system model of the bicycle and directly uses noisy collected data to design the controller. Two models of the bicycle are used to test and assess the controller. Along with the simulations, experiments on a model-scale bicycle demonstrate successful stabilisation using noisy data generated based on the system dynamics.

1 Introduction

Bicycle riding is a frequent human activity and an acquired skill, but creating a controller that can steady a bicycle on its own is a difficult engineering task. **Model Predictive Control (MPC)** and **Linear Quadratic Regulator (LQR)**, two examples of traditional model-based controllers, depend on a precise system model. However, certain aspects of a bicycle, like as the centre of gravity, can be challenging to find and may change as a result of different loads or extra passengers. By solving a **Semi-Definite Programming (SDP)**, a data-driven state-feedback controller is used in this research effort to stabilise a self-balancing minibike. Instead of identifying the bicycle's system model, the method designs the controller using noisy data that has been obtained. To test and evaluate the controller, two bicycle models are used. Using noisy data produced based on the system dynamics, simulations and experiments on a model-scale bicycle show good stabilisation of the system.

To balance a bicycle, a torque should be generated to prevent falling on its side due to gravity. To achieve this, different techniques for torque generation can be introduced such as adding a device to control the leaning, or by controlling the steering angle of the bicycle while moving. While the bicycle is stationary, a flywheel can generate the required torque to achieve the balancing [1]. Another approach is by using a **Control Moment Gyroscope (CMG)** which was tested out by He and Zhao [2]. Since controlling the steering angle to produce the balancing torque does not need additional devices, it was considered by numerous researchers and tested against emerging control algorithms. Balancing using the steering is based on steering into the direction of falling. A drawback of using only the steering angle is that the bicycle needs an initial speed before being able to balance.

This research project is an attempt to apply the theoretical data-driven min-max MPC developed by Xie in [3]. The considered application is a model-scale bicycle at the Institute of System Theory and Automatic Control (IST), University of Stuttgart. The bicycle was chosen as the IST has a rear-steered bicycle which is a more complex system that could be tested against the results achieved from this project. In the following sections, the bicycle modelling and data-driven MPC are discussed.

1.1 Bicycle Modelling

Before implementing a control strategy for balancing a bicycle, a rigorous system model must be established for most techniques, though some approaches are model-

free. System models can be classified as either kinematic or dynamic. The kinematic model does not consider the applied forces on the system, while the dynamic model considers the torques and forces interacting with the system. Researchers set different parameters for the dynamic models depending on the bicycle under study. Some parameters could be neglected to simplify the system. Complex systems emerging from accurate modelling demand sophisticated control algorithms, which require more computation power. Bicycles are non-linear and inherently statically unstable, similar to an inverted pendulum. Examples of dynamic models for the bicycle include the Whipple model [4] and the point mass model [5].

One of the primary methods used in bicycle modelling is the **Linear-Parameter-Varying (LPV)** approach, which accounts for variations in parameters, such as velocity, that affect the stability of the bicycle. The LPV framework is particularly useful when the bicycle's speed is subject to change, as it enables the controller to adapt to varying conditions.

In contrast to kinematic models, kinetic models focus on the fundamental forces and torques on the system. Moreover, software tools for modelling with a multibody approach is used in research such as in [6].

1.2 Bicycle Control

For controlling bicycles, several different approaches were studied by researchers. Andersson et al. [7] proposed a robust **Proportional-Integral-Derivative (PID)** using loop shaping to control a bicycle based on the point mass model. Cerone et al. [8] proposed an LPV-based stabilisation method for riderless bicycles, which uses a parameter-varying approach to handle the dynamic changes in speed. The study demonstrated that stability can be maintained between two specific speeds by discretising the continuous-time system, thereby addressing the nonlinearity inherent in the bicycle dynamics. Tapken et al. [6] studied modelling the kinetic and dynamic aspects of bicycles, employing tools such as CAD modelling and Simscape Multibody to simulate and control the motion of a real mini-bike. Since researchers neglect some parameters in the bicycle models to simplify computations, some uncertainties arise. Therefore, a robust controller can mitigate the errors generated by such models. Robust controllers include the **Sliding Mode Control (SMC)**, which was proposed by Chen and Dao [9], and the **LQR**, which was proposed by Anjumol and Jisha [10]. The controllers proposed in the literature are largely tested in simulations rather than on a real bicycle. This leads to ignoring the emerging uncertainties from real-life models. In [11], Persson et al. conducted a comparative analysis of various control models applied to a driverless bicycle. They studied different PID controllers, an LQR controller,

in addition to a Fuzzy controller. Motion planning for a bicycle was investigated in [12] by solving an optimisation problem, where a constrained Lagrangian is used to model the bicycle as a nonlinear system. A comparative study by Yu et al. [13] investigated the effectiveness of steering control at low speeds, while studying the effect of different trail lengths that introduces the castor effect on the system.

1.3 Data-Driven Control Methods

Data-driven control techniques have emerged as a result of autonomous systems' dynamic and complicated character. These techniques use measurement data directly to infer control actions, without the need for accurate system modelling. When there are unknown process disturbance or model flaws, data-driven approaches are especially beneficial.

One popular method for managing systems with limitations is MPC. Conventional MPC predicts system behaviour using a mathematical model. To reduce modelling errors, data-driven MPC can use input-output data directly to predict future states.

A developed formulation is the data-driven min-max MPC, which aims to robustly stabilise the system while minimizing the worst-case cost. In [3], Xie et al. proposed a robust min-max MPC framework that, using noisy input-state data and formulating the control problem as an SDP, guarantees robust stability and constraint satisfaction while reducing conservatism compared to traditional data-driven MPC approaches. Moreover, in [14], an adaptive scheme for implementing the online data-driven min-max MPC is developed. This is achieved by integrating new data which reduces the model uncertainty over time. The min-max optimisation along with the online adaptation is proved to enhance the robustness. A significant advantage of data-driven MPC is its ability to handle uncertainties without requiring an explicit system model. Compared to classical model-based MPC, data-driven methods are more adaptable when the system's dynamics change. Additionally, the use of online data in adaptive MPC helps reduce conservatism, achieving better closed-loop performance [3].

1.4 WHEELTEC MiniBike

In this research project, a self-balancing mini-bike developed by **WHEELTEC** was utilized as the experimental platform. The goal is to implement a data-driven State-Feedback controller for stabilising the bike. The mini-bike is equipped with various sensors and actuators to enable control and data acquisition. An STM32 Microcontroller is used to control the bike, where the C++ codes used to produce the Hexadecimal file to be burnt on the controller are given with the bicycle.

The bike achieves self-balancing through controlling the steering angle. This is done through a controller computed using LQR combined with a PID controller. The LQR calculates optimal control inputs based on the state feedback to minimize the cost function, while the PID controller fine-tunes the steering speed by adjusting the PWM signals to the servo motor.

While moving with a constant speed, the system's balance is continuously monitored through the gyroscope, which provides real-time data on tilt angles and angular velocities, along with the steering angle. These values are processed by the microcontroller to compute the necessary adjustments to the steering servo to maintain balance.

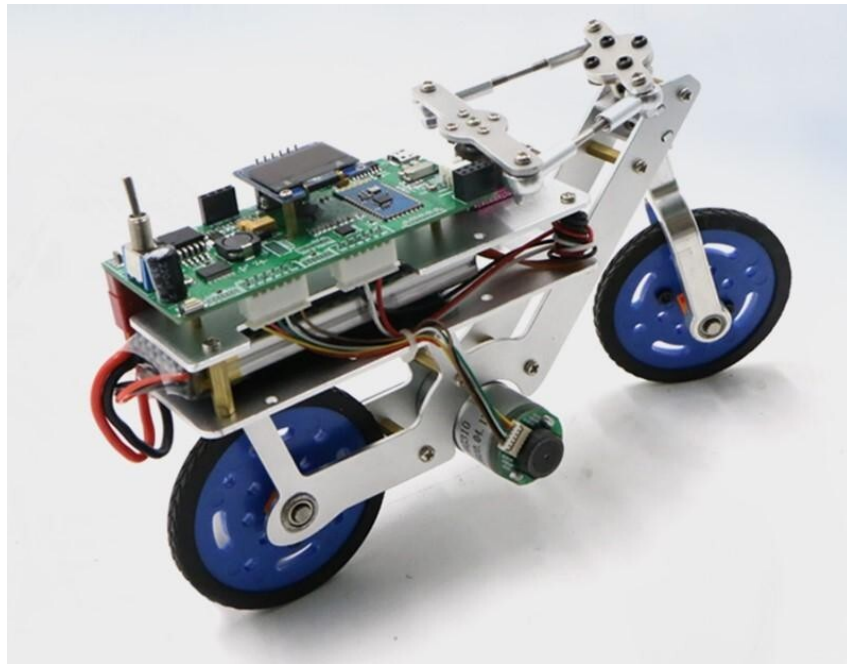


Figure 1.1: A picture for the MiniBike used in the experiments.

1.5 Structure of thesis

This thesis is structured as follows: Chapter 1 provides an introduction, including the background, motivation, literature review, and objectives of the research. In Chapter 2, the theoretical foundations are discussed, focusing on bicycle modelling beside data-driven control techniques. Chapter 3 describes the simulation setup and the implementation of the state-feedback controllers and the simulation results. Chapter 4 presents the experimental work, including the setup and data collection from the mini-bike, along with the results. Chapter 5 concludes the report and discusses future work.

2 Theory

In this section, the theoretical modelling of the bicycle and the data driven Min-Max MPC is discussed.

Since the objective of this research is to simply balance the bicycle, a simple kinetic model is sufficient for this purpose. We use two models and compare between them in terms of control and balance. The difference between both models is considering the fork angle. The fork axis inclination introduces the castor effect which is studied in [13]. For controlling the bicycle, the data-driven MPC is compared with an LQR controller and is applied in simulation and on a real model-scale bicycle to assess the controllers. In this chapter, the bicycle model used is discussed and the theory behind the data-driven state-feedback controller is explained.

2.1 Bicycle Modelling

To model the riderless bicycle, it can be treated as an inverted pendulum. There are some parameters that are neglected such as: elasticity of bicycle parts, the tire-road friction, etc. A simple second-order model of the bicycle can be derived given that the rear-speed is constant, the bicycle rolling on the horizontal plane, and initially, the steer axis λ is in a vertical position as in Figure 2.1. Using the Conservation of Angular Momentum, the following Equation 2.1 is acquired to balance the bicycle [15].

$$\ddot{\varphi} - \frac{g}{h}\varphi = -\frac{av}{hl}\dot{\delta} - \frac{v^2}{hl}\delta, \quad (2.1)$$

where φ is the lean angle, δ is the steer angle, g is the gravity acceleration, h is the height of the center of mass, l is the wheelbase, a is the distance from the center of mass to the front wheel and v is the speed of the bicycle. The bicycle model is a second-order system with two states: lean angle φ and steer angle δ . This is based on linearisation where the desired stability steady-state for both angles δ and ϕ is 0.

Now, to consider the fork angle λ , which is shown in Figure 2.2, Equation (2.1) is expanded as follows.

$$\ddot{\varphi} - \frac{g}{h}\varphi = -\frac{av \sin \lambda}{hl}\dot{\delta} - \frac{(v^2h - ar_\tau g) \sin \lambda}{h^2l}\delta, \quad (2.2)$$

where λ is the fork angle, and r_τ is the trail distance such that $r_\tau = \tan(\frac{\pi}{2} - \lambda)$. It is clear that the difference between both (2.1) and (2.2) is putting the fork angle $\lambda = 90^\circ$

and $r_\tau = 0$. The state-space representation of the system model considering the fork angle given by Equation 2.2 is

$$\begin{bmatrix} \dot{\varphi} \\ \ddot{\varphi} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ \frac{g}{h} & 0 & \frac{(ar_\tau g - v^2 h) \sin \lambda}{h^2 l} \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \varphi \\ \dot{\varphi} \\ \delta \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{-av \sin \lambda}{hl} \\ 1 \end{bmatrix} \dot{\delta}. \quad (2.3)$$

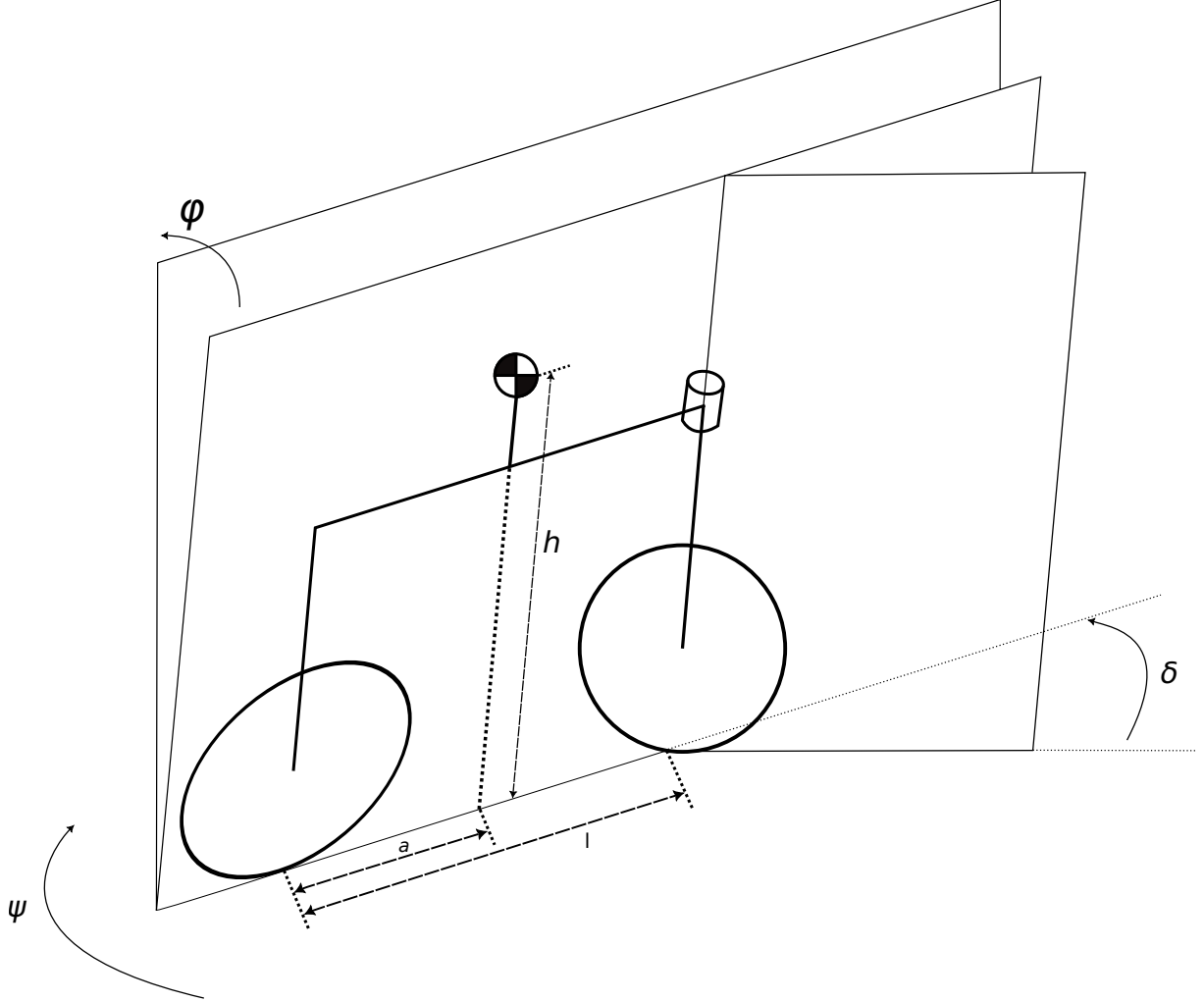


Figure 2.1: A Kinetic no-fork model of the bicycle

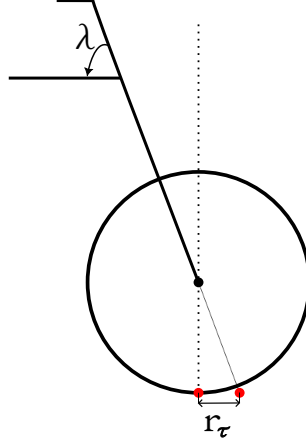


Figure 2.2: A fork model showing an inclination with angle λ of the steering axis, introducing a trail distance r_{tau} .

2.2 Data-Driven Min-Max MPC

In this section, we show how to derive the Data-Driven Min-Max MPC mentioned in [3]. To formulate our problem, we consider a discrete-time Linear Time-Invariant (LTI) system affected by process noise:

$$x_{t+1} = A_s x_t + B_s u_t + \omega_t, \quad (2.4)$$

where $x_t, x_{t+1} \in \mathbb{R}^n$ are the state vectors, $u_t \in \mathbb{R}^m$ is the control input, and $\omega_t \in \mathbb{R}^n$ represents unknown but bounded disturbance at time t . The system matrices A_s and B_s are assumed to be unknown. Instead of identifying a model explicitly, we utilize available noisy input-state data to design a stabilizing controller. The noise ω_t is assumed to satisfy a known upper bound $\epsilon \geq 0$ such that

$$\|\omega_t\|_2^2 \leq \epsilon.$$

To move to the data-driven framework, a data sequence consisting of states x_i , inputs u_i , and process noise ω_i where $i \in \mathbb{N}$ is considered. The system in Equation 2.4 can then be rewritten in terms of the data sequence as

$$\omega_i = x_{i+1} - A_s x_i - B_s u_i, \quad (2.5)$$

where $i \in \mathbb{N}$ denotes the data index. Let the pair (A, B) be the set of system matrices that satisfy Equation 2.5 and the noise bound. This set of system matrices can be

written as the following quadratic matrix inequality

$$\Sigma_i = \left\{ (A, B) : \begin{bmatrix} I & A & B \end{bmatrix} \begin{bmatrix} I & x_{i+1} \\ 0 & -x_i \\ 0 & -u_i \end{bmatrix} \begin{bmatrix} \epsilon I & 0 \\ 0 & -I \end{bmatrix} \begin{bmatrix} I & 0 & 0 \\ x_{i+1} & -x_i & -u_i \end{bmatrix} \begin{bmatrix} I \\ A \\ B \end{bmatrix} \succeq 0 \right\}. \quad (2.6)$$

Then, the set of system matrices consistent with all data points can be described as follows

$$\mathcal{C} = \bigcap_{i=0}^{T-1} \Sigma_i, \quad (2.7)$$

which is equivalent to the following quadratic matrix inequality

$$\mathcal{C} = \left\{ (A, B) : \begin{bmatrix} I & A & B \end{bmatrix} \Pi(\tau) \begin{bmatrix} I & A & B \end{bmatrix}^\top \succeq 0, \forall \tau = (\tau_0, \dots, \tau_{T-1}), \tau_i \geq 0, i \in \mathbb{I}_{[0, T-1]} \right\}, \quad (2.8)$$

where $\Pi(\tau)$ is

$$\Pi(\tau) = \sum_{i=0}^{T-1} \tau_i \begin{bmatrix} I & x_{i+1} \\ 0 & -x_i \\ 0 & -u_i \end{bmatrix} \begin{bmatrix} \epsilon I & 0 \\ 0 & -I \end{bmatrix} \begin{bmatrix} I & x_{i+1} \\ 0 & -x_i \\ 0 & -u_i \end{bmatrix}^\top. \quad (2.9)$$

To design a control law that stabilises the system despite the uncertainty in (A, B) , we setup the problem by defining a quadratic stage cost function as

$$l(u, x) = u^\top R u + x^\top Q x,$$

where $\ell(x, u) = x^\top Q x + u^\top R u$ is the quadratic stage cost, with weighting matrices $Q \succ 0$ and $R \succ 0$. such that the weighting matrices $R, Q \succ 0$. In addition, we define state and input ellipsoidal constraints by the following

$$\|u_t\|_{S_u} \leq 1, \quad \|x_t\|_{S_x} \leq 1, \quad \forall t \in \mathbb{N},$$

where $S_u \succ 0$, $S_x \succeq 0$, and $\|x\|_P = \sqrt{x^\top P x}$. Now, we can state a data-driven min-max MPC problem at time t by

$$J_{\infty}^*(x_t) := \min_{\bar{u}(t)} \max_{(A,B) \in \mathcal{C}} \sum_{k=0}^{\infty} l(\bar{u}_k(t), \bar{x}_k(t)) \quad (2.10a)$$

$$\text{s.t. } \bar{x}_{k+1}(t) = A\bar{x}_k(t) + B\bar{u}_k(t), \quad (2.10b)$$

$$\bar{x}_0(t) = x_t, \quad (2.10c)$$

$$\|\bar{u}_k(t)\|_{S_u} \leq 1, \forall t \in \mathbb{N}, \quad (2.10d)$$

$$\|\bar{x}_k(t)\|_{S_x} \leq 1, \forall (A,B) \in \mathcal{C}, t \in \mathbb{N}. \quad (2.10e)$$

This cost function can be explained by being a minimisation of the worst-case cost l over all system matrices in $\mathcal{C}$ by adjusting the input $\bar{u}_k(t), \forall k \in \mathbb{N}$ to the system. $\bar{x}_k(t)$ is the predicted state and $\bar{u}_k(t)$ is the control input, both at time step $t+k$. The sets (A,B) used are consistent with the system dynamics in (2.10b). We let the initial state be the measurement state x_t as shown in constraint (2.10c). The ellipsoidal constraints for the control input and states are given in constraints (2.10d) and (2.10e) respectively.

Now, the data-driven min-max MPC can be simplified by just considering a state-feedback control law, such that $u_t = F_t x_t$, where $F_t \in \mathbb{R}^{m \times n}$.

In [3], it is shown that the data-driven min-max MPC in Equation 2.10 can be reformulated as an SDP. Suppose that there exist $\gamma > 0$, $H \in \mathbb{R}^{n \times n}$, $L \in \mathbb{R}^{m \times n}$, $\tau \in \mathbb{R}^T$. The SDP problem is expressed as the following

$$\underset{\gamma, H, L, \tau}{\text{minimise}} \gamma \quad (2.11a)$$

s.t. the following LMIs hold

$$\begin{bmatrix} 1 & x_t^\top \\ x_t & H \end{bmatrix} \succeq 0, \quad (2.11b)$$

$$\begin{bmatrix} \begin{bmatrix} -H & 0 \\ 0 & 0 \end{bmatrix} + \Pi(\tau) & \begin{bmatrix} 0 \\ H \\ L \end{bmatrix} & 0 \\ \begin{bmatrix} 0 & H & L^\top \\ 0 & 0 & 0 \end{bmatrix} & -H & \Phi^\top \\ & \Phi & -\gamma I \end{bmatrix} \prec 0, \quad (2.11c)$$

$$\tau = (\tau_0, \dots, \tau_{T-1}), \tau_i \geq 0, \forall i \in \mathbb{I}_{[0, T-1]}, \quad (2.11d)$$

$$\begin{bmatrix} H & L^\top \\ L & S_u^{-1} \end{bmatrix} \succeq 0, \quad (2.11e)$$

$$\begin{bmatrix} S_x & I \\ I & H \end{bmatrix} \succeq 0. \quad (2.11f)$$

where $\Phi = \begin{bmatrix} M_R L \\ M_Q H \end{bmatrix}$ and $M_R^\top M_R = R$, $M_Q^\top M_Q = Q$. The γ upper bounds the optimal cost in Equation 2.10. The LMIs (2.11b), (2.11c), and (2.11d) ensure that the system dynamics with the bounded noise are satisfied. While the LMIs (2.11e) and (2.11f) define the state and input constraints. The optimal solution for the problem (2.11) is then expressed as γ^* , H^* , L^* , τ^* . Consequently, the optimal state-feedback gain is $F = L^*(H^*)^{-1}$.

The SDP problem is then solved in a receding horizon manner as per Algorithm 1. At a time-step t , (2.11) is solved and the state-feedback gain F_t is acquired and then the first input $u_t = F_t x_t$ is applied.

Algorithm 1 Data-driven min-max MPC

- 1: At time $t = 0$, measure state x_0 ;
 - 2: Solve the problem (2.11);
 - 3: Apply the input $u_t = F_t x_t$;
 - 4: Set $t = t + 1$ and go back to 2;
-

3 Simulations

To assess the performance of different controllers for stabilising the bicycle, different simulations were conducted. The two bicycle models from Section 2.1 are considered:

- **Simple Model:** A simplified kinetic model assuming the fork axis is vertical.
- **Inclined Fork Model:** A kinetic model including the fork angle, accounting for the steering head geometry.

Two versions of the data-driven state-feedback controllers were tested. The first version is a data-driven min-max by solving the SDP in Equation (2.11) only once for an initial state. While the second version is a data-driven min-max MPC where the SDP is solved at each time step. To evaluate the data-driven control laws performance, they are compared to a state-feedback LQR controller. Additionally, identical process noise was introduced to all simulations.

The simulations were performed using MATLAB based on the two kinetic models. The following assumptions were made:

- The bicycle moves on a horizontal plane.
- The rear wheel maintains a constant forward velocity.
- Small-angle approximations are applied for the lean angle (ϕ) and steering angle (δ).

The key physical parameters used are summarized in Table 3.1. The open-loop system controllability is guaranteed before deriving the controllers. Moreover, the closed-loop stability of the system is ensured by checking if the eigenvalues are strictly within the unit circle of the complex plane.

Parameter	Value	Unit	Description
h	0.088	m	Height of center of mass
a	0.055	m	Distance between rear wheel and centre of gravity projection
λ	75 or 90	degrees	Fork angle: angle between the steering axis and the horizontal line
r_w	0.0375	m	Radius of the wheel
r_τ	$r_w \cdot \tan\left(\frac{\pi}{2} - \lambda\right)$	m	Trail Distance: distance between front wheel ground contact point and intersection of fork with ground
g	9.81	m/s ²	Gravitational acceleration
v	0.634	m/s	Forward speed
w	0.167	m	Distance between front and rear wheels at ground contact points

Table 3.1: Simulation Parameters

The state constraints for the bicycle system were given by

$$\begin{aligned}
|\varphi| &\leq 35^\circ, \\
|\dot{\varphi}| &\leq 100^\circ, \\
|\delta| &\leq 40^\circ.
\end{aligned}$$

While the input constraint was given by

$$|\dot{\delta}| \leq 750^\circ.$$

Accordingly, the state and input constraints are defined by S_x and S_u respectively as follows:

$$S_x = \begin{bmatrix} \frac{1}{\varphi_{max}^2} & 0 & 0 \\ 0 & \frac{1}{\dot{\varphi}_{max}^2} & 0 \\ 0 & 0 & \frac{1}{\delta_{max}^2} \end{bmatrix} \approx \begin{bmatrix} 2.6798 & 0 & 0 \\ 0 & 0.32828 & 0 \\ 0 & 0 & 2.05 \end{bmatrix},$$

$$S_u = \frac{1}{\dot{\delta}_{max}^2} \approx 0.005836.$$

Three different simulations corresponding to different control strategies are discussed below.

3.1 Linear Quadratic Regulator (LQR)

An LQR controller was tested against both the fork and no-fork bicycle models by solving the discrete-time algebraic Riccati equation. Different weighting matrices Q and R were selected to modify the weights given to the states and inputs respectively, where the weights are used to compute the closed loop cost.

The LQR served as a baseline controller and to generate input-state data necessary for the subsequent data-driven methods. Figure 3.1 shows two plots for different comparisons. In Figure 3.1a, four simulations are conducted to show the difference in performance for different Q and R settings. It shows that giving more weight to the second state, namely the lean angle rate of change, smooths the lean angle response and reduces its steepness, while having a slower reaction in terms of balancing. Since the input is the steering angle rate of change $\dot{\delta}$, giving it more weight smoothens all states. For the other simulations and experiments with different controllers, the Q and R corresponding to the fastest response here are used. In Figure 3.1b, both bicycle models responses are compared, where it shows that the vertical fork model has a lower overshoot in the steering state as well as having a lower closed loop cost as shown in Table 3.2.

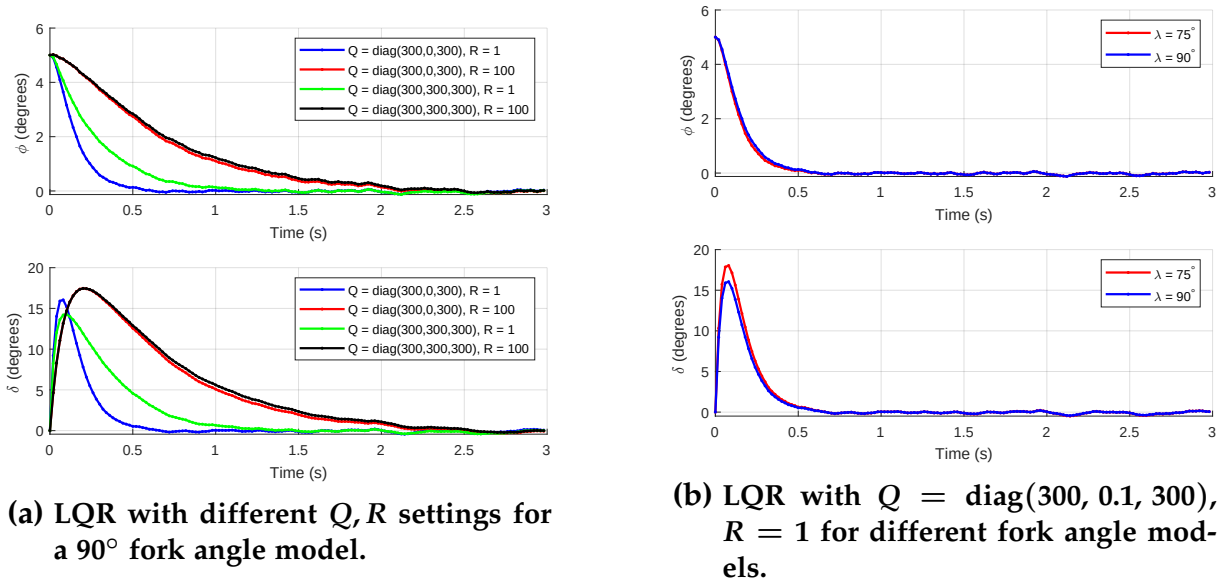


Figure 3.1: Lean and steering angles over time under LQR controller with different Q and R settings and different system models, for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, and process noise bounded by $\epsilon = 10^{-6}$, and sampling period $T_s = 0.02s$.

Fork Angle	Q, R Settings	Cumulative Cost
75°	$Q = \text{diag}(300, 0.1, 300), R = 1$	341.8579
90°	$Q = \text{diag}(300, 0.1, 300), R = 1$	273.3007
90°	$Q = \text{diag}(300, 0.1, 300), R = 1$	273.3007
90°	$Q = \text{diag}(300, 300, 300), R = 1$	546.1975
90°	$Q = \text{diag}(300, 0.1, 300), R = 100$	5044.3065
90°	$Q = \text{diag}(300, 300, 300), R = 100$	5130.1350

Table 3.2: Comparison of LQR controller performance across different Q, R settings and fork models

3.2 Data-Driven State-Feedback Controller

Using generated input-state data, a data-driven state-feedback controller was constructed by solving the SDP problem (2.11) once and applying the acquired control gain. The controller is obtained without knowledge of the exact system model, relying solely on the data and solving the optimisation problem as mentioned in Section 2.2.

A range of data is synthesised in simulation given an initial starting point for the system, an error bound for adding noise, and being controlled with a discrete LQR that stabilises the bicycle. The synthesised data formed a sequence where at each time step it was distorted with additive process noise $\omega_i \in \mathbb{R}^n$ with independent components uniformly sampled as

$$\omega_i^j \sim \mathcal{U} \left[-\frac{\epsilon}{\sqrt{n}}, \frac{\epsilon}{\sqrt{n}} \right], \quad j = 1, \dots, n,$$

which can be compactly written as

$$\omega_i \sim \mathcal{U} \left[-\frac{\epsilon}{\sqrt{n}}, \frac{\epsilon}{\sqrt{n}} \right]^n$$

to satisfy $\|\omega\|_2^2 \leq \epsilon$.

The data-driven state-feedback controller based on this data stabilised the bicycle efficiently and computed a controller gain similar to that acquired by the discretised LQR.

Later, the output feedback gain vector from the LQR was applied on the real bicycle to collect some real data, instead of the synthesised data, for simulating and testing the data-driven state-feedback controllers. It turned out not to work since the process noise ϵ was too large for various segments of the data collected. From this, it was obvious that an online version of the data-driven state-feedback controllers would not work. This challenge is based on the real system errors compared to the synthesised data errors which were always bounded as needed.

In Figure 3.2, the simulation results for different fork angles under identical settings are presented. The feedback gains are comparable to those obtained from the LQR controllers, demonstrating that the controller is capable of balancing the bicycle.

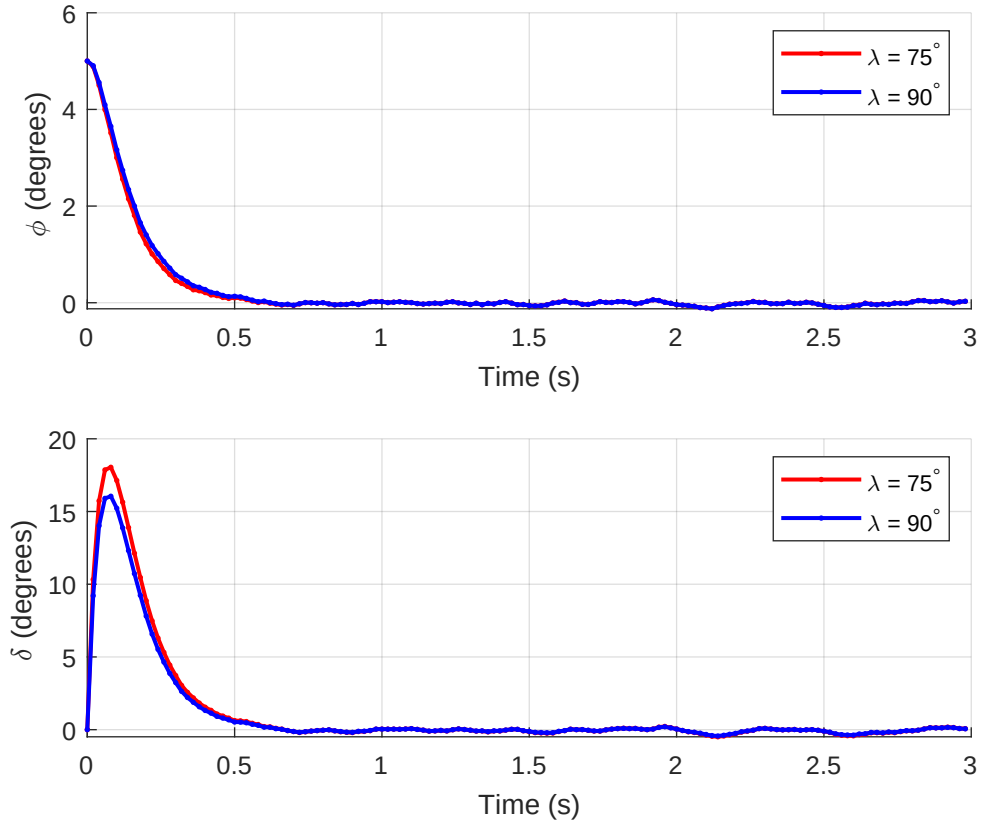


Figure 3.2: Lean and steering angles over time under different data-driven state-feedback settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, process noise bound $\epsilon = 10^{-6}$ and sampling period $T_s = 0.02s$ for different fork angle models.

3.3 Data-Driven Min-Max MPC Controller

Finally, a full data-driven min-max MPC controller was implemented using synthesised input-state data similar to the previous controller. Here, the SDP (2.11) was solved in a receding horizon manner.

The simulation results for different fork angles under the same parameters are shown in Figure 3.3. The error bound ϵ is decreased to 10^{-5} to ensure obtaining a solution for the SDP at each iteration of the MPC.

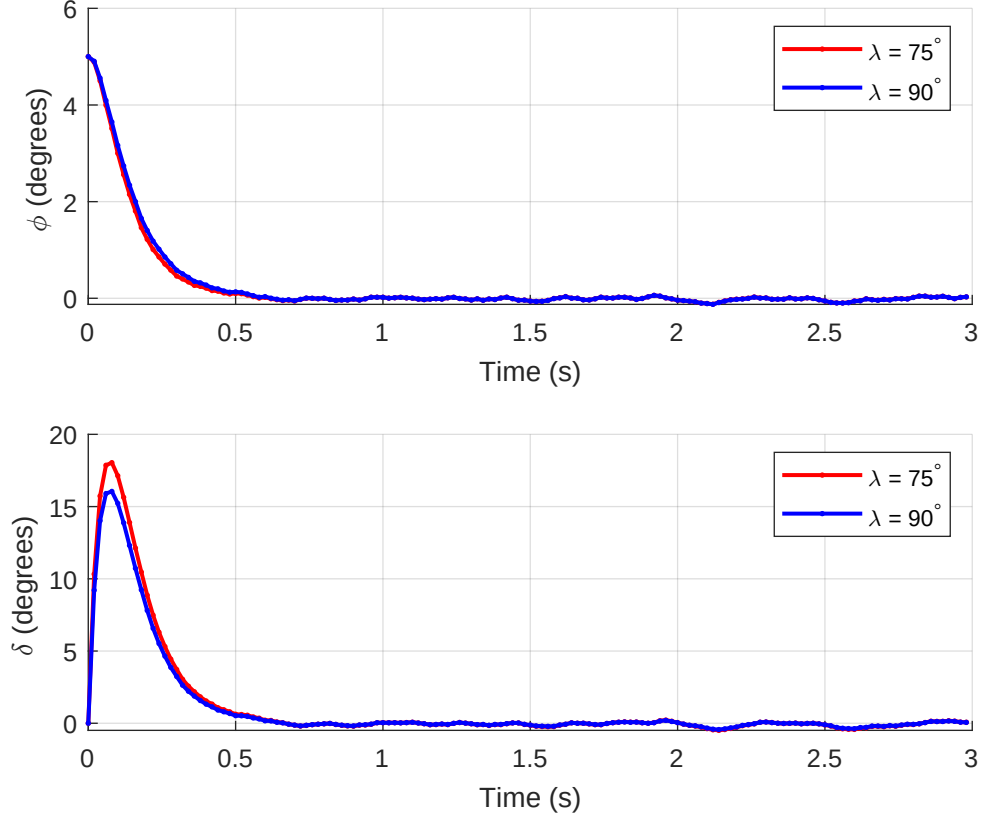


Figure 3.3: Lean and steering angles over time under different data-driven MPC settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^T$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, noise bound $\epsilon = 10^{-6}$, sampling period $T_s = 0.02s$, and 150 MPC iterations for different fork angle models.

3.4 Quantitative comparison of different controllers

The three controllers performance was assessed by computing the closed loop costs under the same Q, R settings and identical process noise for all simulations, as shown in Table 3.3. Since the error bound for both the synthesised data used in the data-driven techniques and the process noise was low, the costs for the data-driven controllers were less than the LQR controller cost. This also shows the robustness of the data-driven controllers used under process noise.

Controller	Fork angle	Cost / Performance Metric
LQR	75°	341.8579
Data-Driven State-Feedback	75°	341.8560
Data-Driven Min-Max MPC	75°	341.8560
LQR	90°	273.3007
Data-Driven State-Feedback	90°	273.2988
Data-Driven Min-Max MPC	90°	273.2988

Table 3.3: Comparison of controllers across different fork models under $Q = \text{diag}(300, 0.1, 300)$, $R = 1$ and data length of $T = 120$

4 Experimental Work

4.1 Mini-Bike

The mini-bike used in the experiments based on the **WHEELTEC Z100 Servo Self-Balancing Bicycle** in Figure 4.1. It features an STM32F103C8T6 microcontroller as the control unit, which utilises different sensors and motor control circuits to achieve stabilisation. A similar bicycle was studied in [6] using multibody software modelling.

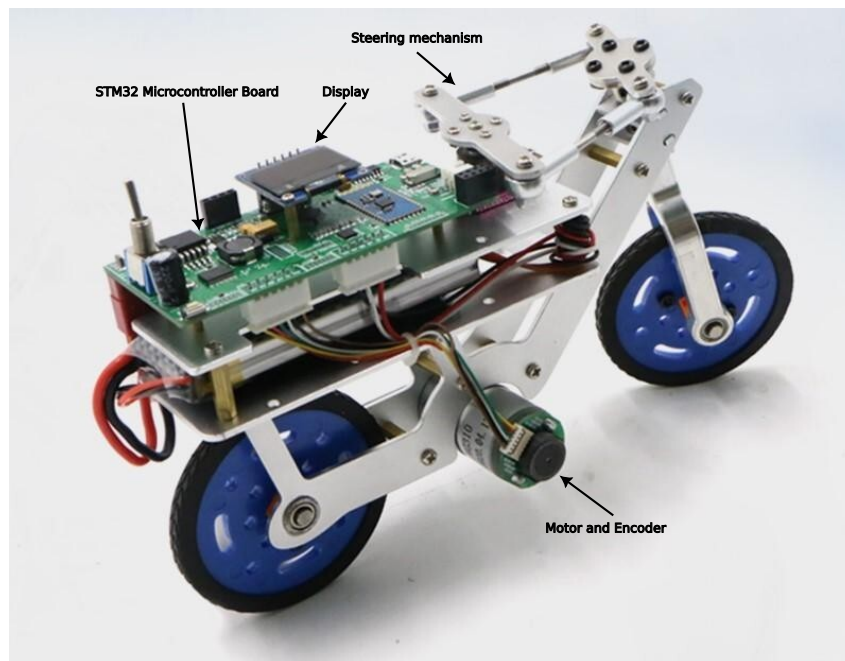


Figure 4.1: A picture for the MiniBike used in the experiments.

The bike has the following key components:

- **Microcontroller:** STM32F103C8T6 (ARM Cortex-M3)
- **IMU Sensor:** MPU6050 (3-axis gyroscope and accelerometer)
- **Servo Motor:** WHEELTEC Z100 steering servo
- **Rear Wheel Motor:** DC motor with incremental encoder
- **Motor Driver:** AT8236 H-bridge circuit
- **Power Supply:** 2S lithium-ion battery (7–12 V)
- **Wireless Communication:** Bluetooth module (USART3)

The bike uses steering angle control to self-balance. This is done through an control gain, originally from LQR, followed by a PID controller. The LQR computes the state-

feedback gain that minimises a cost function. While the PID controller adjusts the PWM signals to the servo motor to fine-tune the steering speed.

The bike is balanced continuously through getting data from the gyroscope, which provides real-time data on tilt angles and angular velocities. The measurements are processed by the microcontroller to compute the control law, where the input is the steering angle derivative. The microcontroller is configured to send control inputs to the servo at a frequency of 50 Hz.

The Data was collected from the bike through a Bluetooth connection using US-ART3, where the frequency of data was about 10 Hz. A mobile device was connected to the bike where the data received was saved for processing.

The mini-bike allows adjustment of the balance midpoint through a potentiometer; if it is offset from zero, the bike will move in a circle in the direction of the lean. Moving in a circle allowed for a longer range of data collected while driving under a controller.

The control algorithms were implemented on the STM32 using C++ programming with HAL Drivers. The platform on which the development of codes were done is the *STM32Cube IDE*, which is a free unlimited software for writing and compiling the codes. It compiles the codes into a Hexadecimal file which can then be flashed on the STM32 microcontroller using FlyMCU software.

The codes read the IMU data (lean angle and angular velocity) via I2C communication. Then, the forward velocity is estimated by processing the encoder pulses. The lean angle is estimated by a complementary filter. The implemented control law is then applied using the states and a control input is determined. Consequently, the PWM signal of the steering servo is computed and applied.

The data-driven MPC controller was not implemented since the real-life errors prevented successful solving of the SDP on real data as stated before. Due to this limitation, the SDP optimisation problem was solved offline, and the resulting state-feedback gain was coded into the embedded controller as a static gain.

The system states that were recorded are the lean angle (ϕ), lean rate ($\dot{\phi}$), and Steering angle (δ).

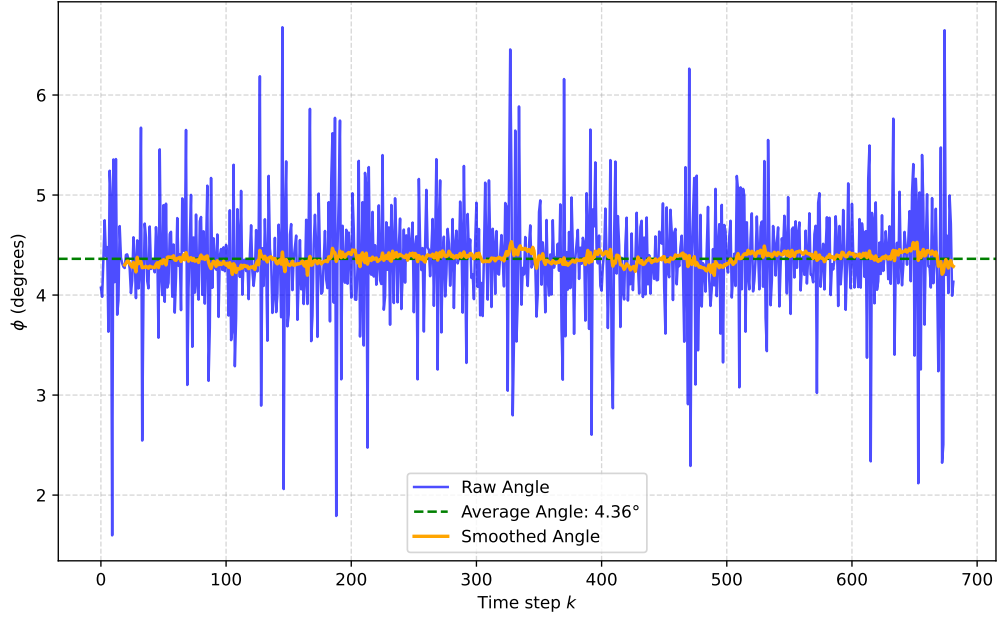
4.2 Experiments

Four different controllers were applied on the mini-bike based on both the LQR and data-driven state-feedback discussed in Chapter 3. The experiments are described in Table 4.1.

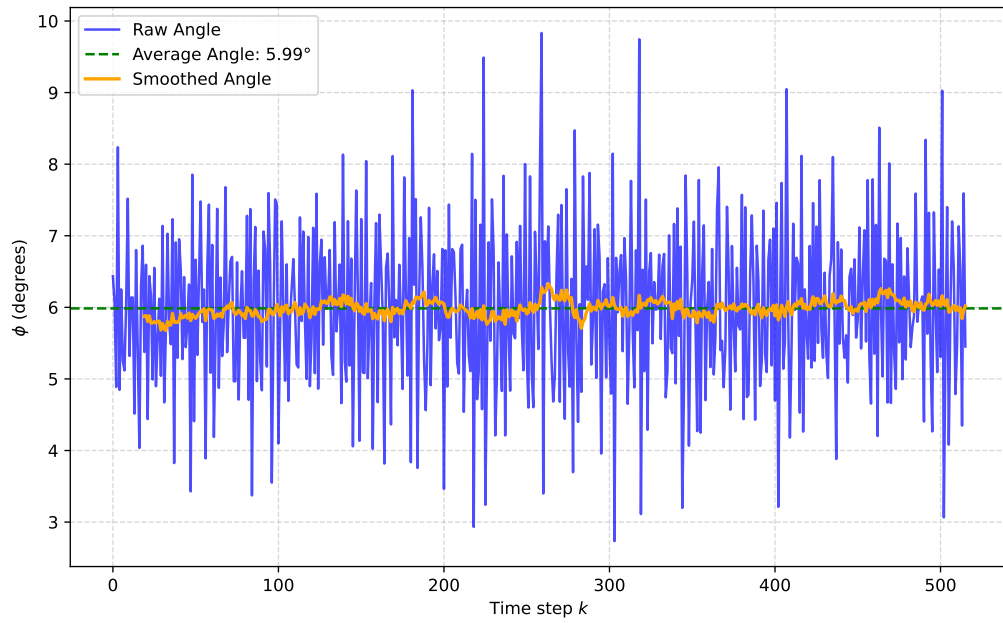
Experiment	Controller	Fork Angle	Target Angle
1	LQR	75°	4°
2	LQR	90°	5°
3	Data-driven state-feedback	75°	3.5°
4	Data-driven state-feedback	90°	4°

Table 4.1: Overview of controllers applied on the mini-bike

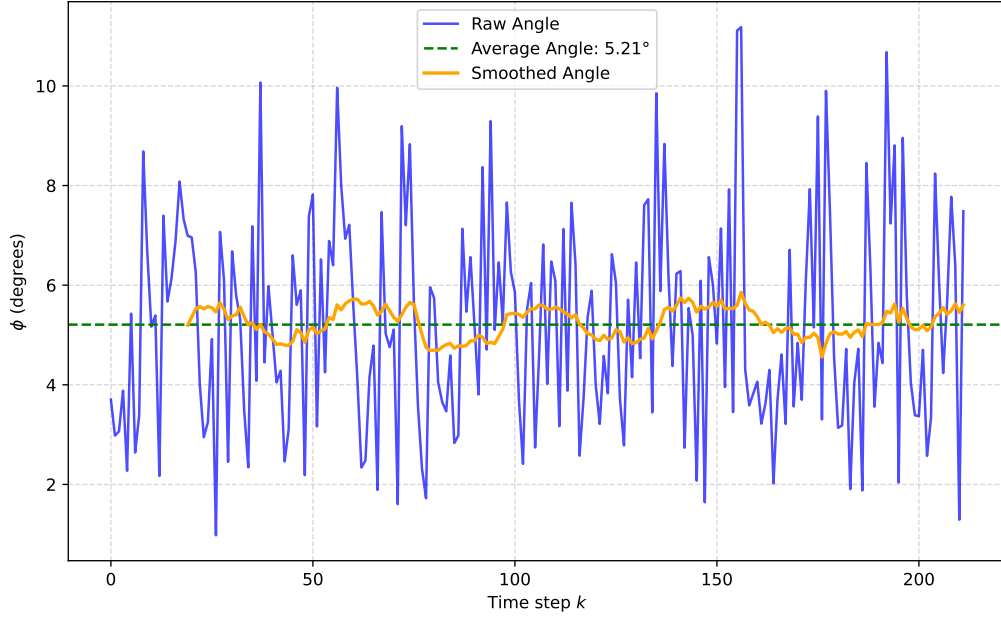
The results acquired on implementing the controllers and collecting the data are plotted in Figure 4.2. It can be noticed that the data length varies across the experiments. This is due to the varying capability of collecting the data before the bicycle falls, particularly in experiment 3 in Figure 4.2c, where the controller did not maintain bicycle balance for an extended period. In Figure 4.2d of Experiment 4, it is noticed that the data-driven state-feedback on the fork angle bicycle model achieved balancing of the bicycle comparable to that of the LQR. Figures 4.2 show that the average lean angle achieved differs from the target angle. This can arise from different factors including the external noise and inaccuracies in measuring the potentiometer value responsible for setting the target angle.



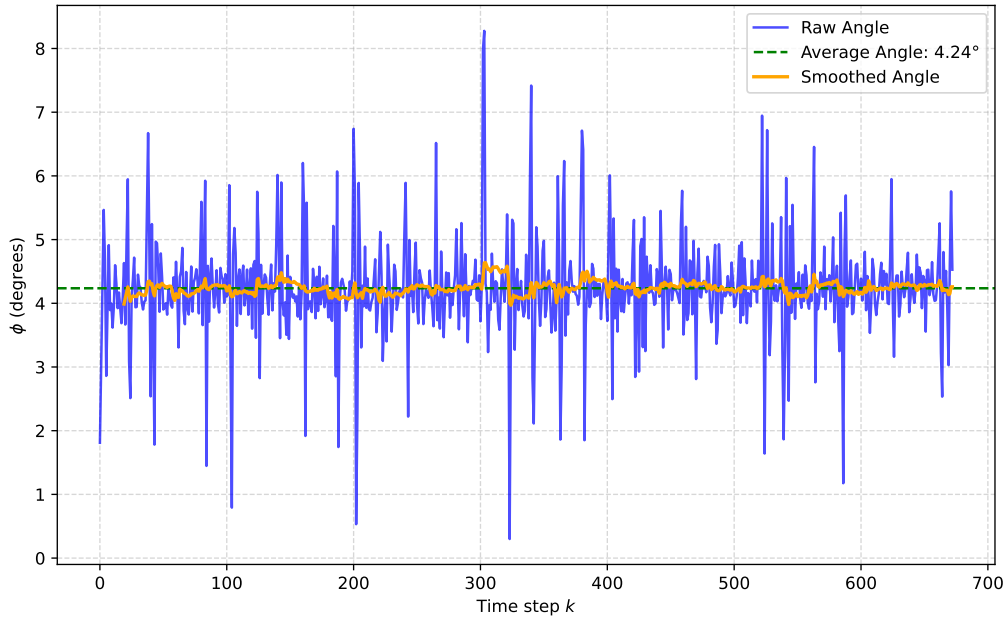
(a) LQR for 75° fork angle and 4° target angle.



(b) LQR for 90° fork angle and 5° target angle.



(c) Data-driven state-feedback for 75° fork angle and 3.5° target angle.



(d) Data-driven state-feedback for 75° fork angle and 4° target angle.

Figure 4.2: Lean angle over time under different data-driven state-feedback settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, and noise bound $\epsilon = 0.001$ for data synthesis of length $T = 40$. The data is smoothed using a rolling window which takes the mean of each 20 data points. The average lean angle achieved is also shown for each experiment.

5 Conclusion and Future work

5.1 Conclusion

In this research project, the goal was to implement a data-driven MPC and test it for balancing a bicycle. Two models for the bicycle were studied which were simple second-order systems with and without considering the fork axis angle. The theoretical basis for the data-driven controller was studied and discussed where the data-driven min-max MPC was derived, narrowed down to state-feedback, then reformulated to be solved as an SDP in a sliding horizon manner. Following that, the data-driven controller was implemented in Matlab, modified for the models of the bicycle along with the input and state constraints, and then simulated for two versions of the data-driven state-feedback controller. The first version solved the SDP once, and then the controller was passed to the bicycle model in simulation. The second version solved the SDP as an MPC problem with shifting horizon. The real-life experiments were then carried on the WHEELTEC Mini-Bike by implementing four controllers, an LQR controller with two versions for both the fork and no-fork angle model of the bicycle, in addition to a data-driven state-feedback based on a one-shot solution of the SDP was applied on the bicycle with its two model versions.

5.2 Future Work

Several points could be encountered for future work including:

- decreasing the errors in the collected data for feeding the data-driven controllers,
- add an adaptive scheme for implementing the online data-driven min-max MPC to increase robustness as in [14],
- exploring the linear parameter-varying system with a variable speed for the bicycle,
- exploring other complex systems such as a rear-steered bicycle.

Bibliography

- [1] Masahiro Kawaguchi and Masaki Yamakita. “Stabilizing of bike robot with variable configured balancer”. In: *SICE Annual Conference 2011*. 2011, pp. 1057–1062.
- [2] Jiarui He and Mingguo Zhao. “Control System Design of Self-balanced Bicycles by Control Moment Gyroscope”. In: *Proceedings of the 2015 Chinese Intelligent Automation Conference*. Ed. by Zhidong Deng and Hongbo Li. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 205–214. ISBN: 978-3-662-46466-3.
- [3] Yifan Xie, Julian Berberich, and Frank Allgöwer. “Data-Driven Min-Max MPC for Linear Systems”. In: *2024 American Control Conference (ACC)*. 2024, pp. 3184–3189. DOI: 10.23919/ACC60939.2024.10644295.
- [4] F. J. W. Whipple. “The stability of the motion of a bicycle”. In: *Quarterly Journal of Pure and Applied Mathematics* 30 (1899), pp. 312–348.
- [5] D.J.N. Limebeer and R.S. Sharp. “Bicycles, motorcycles, and models”. In: *IEEE Control Systems Magazine* 26.5 (2006), pp. 34–61. DOI: 10.1109/MCS.2006.1700044.
- [6] C. Tapken, A. Gloesekoetter, and J. Gloesekoetter. “Modelling and Controlling the Kinetic and Dynamic of a Bicycle”. In: *Enseñanza y Aprendizaje de Ingeniería de Computadores* 8 (2018), pp. 97–113. URL: https://digibug.ugr.es/bitstream/10481/53302/1/T8_N8_Revista_EAIC_2018.pdf.
- [7] Tom Andersson et al. “A Loop Shaping Method for Stabilising a Riderless Bicycle”. In: *2019 European Conference on Mobile Robots (ECMR)*. 2019, pp. 1–6. DOI: 10.1109/ECMR.2019.8870965.
- [8] Vito Cerone, Davide Andreo, and Mats Larsson. “Stabilization of a Riderless Bicycle A Linear-Parameter-Varying Approach”. In: URL: <https://api.semanticscholar.org/CorpusID:14769320>.
- [9] Chih-Keng Chen and Trung-Kien Dao. “A study of bicycle dynamics via system identification”. In: *2010 International Symposium on Computer, Communication, Control and Automation (3CA)*. Vol. 2. 2010, pp. 204–207. DOI: 10.1109/3CA.2010.5533583.
- [10] M. A. Anjumol and V. R. Jisha. “Optimal stabilization and straight line tracking of an electric bicycle”. In: *2014 International Conference on Power Signals Control and Computations (EPSCICON)*. 2014, pp. 1–6. DOI: 10.1109/EPSCICON.2014.6887487.

- [11] Niklas Persson et al. "A Comparative Analysis and Design of Controllers for Autonomous Bicycles". In: *Proceedings of the 2021 European Control Conference (ECC)*. IEEE, 2021, pp. 1570–1576. DOI: 10.23919/ECC54610.2021.9655083. URL: https://www.es.mdu.se/pdf_publications/6175.pdf.
- [12] Alen Turnwald and Steven Liu. "Motion Planning and Experimental Validation for an Autonomous Bicycle". In: *IECON 2019 - 45th Annual Conference of the IEEE Industrial Electronics Society*. Vol. 1. 2019, pp. 3287–3292. DOI: 10.1109/IECON.2019.8927829.
- [13] Yongchao Yu and Mingguo Zhao. "Steering Control for Autonomously Balancing Bicycle at Low Speed". In: *2018 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. 2018, pp. 33–38. DOI: 10.1109/ROBIO.2018.8665347.
- [14] Yifan Xie, Julian Berberich, and Frank Allgöwer. "Data-Driven Min-Max MPC for Linear Systems: Robustness and Adaptation". In: *arXiv preprint arXiv:2404.19096* (2024). URL: <https://arxiv.org/abs/2404.19096>.
- [15] Karl J. Åström, Richard E. Klein, and Anders Lennartsson. "Bicycle Dynamics and Control". In: *IEEE Control Systems Magazine* 25.4 (2005), pp. 26–47. DOI: 10.1109/MCS.2005.1499389.